

OrbitFaaS: Deterministic Scale-to-Zero Federated Learning for LEO Satellite Constellations

Principal Investigator: Prof. Bharat Bhargava, Department of Computer Science, Purdue University
Contact Email: bbshail@purdue.edu **Phone Number:** +1 (765) 413-7312

Abstract

LEO constellations collect more imagery than they can downlink. A satellite in low Earth orbit completes a revolution in roughly 90 to 120 minutes and is in view of any one ground station for 5 to 15 minutes of it. Federated learning (FL) fits this regime: satellites train on the data they hold and exchange only model updates. Every satellite FL system published so far, however, runs on a substrate built for datacenters, with a parameter server that stays up between contacts and a training runtime resident on every satellite. Both waste memory and harvested energy that a small satellite cannot spare. Serverless computing, event-driven and scale-to-zero, matches the duty cycle of an orbital node far better, yet it has been studied for FL only in terrestrial clouds, where clients are reachable, bandwidth is stable, and compute is elastic. None of these holds in orbit.

We propose OrbitFaaS, a serverless FL orchestration framework built on the one property LEO has that terrestrial clouds lack: contact times are known in advance. The project has three research questions. A visibility-aware scheduler turns cold-start management from prediction into deterministic scheduling using SGP4 propagation of public TLEs, with a keep-warm policy we prove optimal under deterministic arrivals. A measurement study quantifies where aggregation state should live under orbital churn and yields a hybrid placement policy. OrbitPath-FL decides, per handoff, which training state is worth carrying when a satellite leaves visibility mid-round, backed by a convergence analysis under non-IID data. We evaluate on a testbed that drives a real serverless backend and a real FL framework from orbital simulation, using EuroSAT and one large Earth-observation dataset. The PI's group has published a satellite-terrestrial FL paper, and the student lead has built and run the FedLess serverless FL framework.

1 Background & Motivation

Sony already flies cameras and optical links. The STAR SPHERE project's EYE satellite, a 6U CubeSat developed with the University of Tokyo and JAXA and launched in January 2023, images Earth from about 500 km through a Sony full-frame sensor and a 28-135 mm f/4 lens. Sony CSL's SOLISS terminal on the ISS closed a bidirectional 1.5 μm laser link to an optical ground station in March 2020 and carried Ethernet traffic over it at 100 Mbps [1], and the SOL roadmap extends that link into a layered optical network of space-to-ground links, inter-satellite cross-links (ISLs), and relay through HAPS and GEO; Sony Space Communications Corporation, founded in 2022, is building small terminals to connect microsatellites in LEO by laser. Sony Semiconductor image sensors are standard parts in Earth-observation payloads, and Sony Semiconductor Israel's ALT1250 modem supports 3GPP Release 17 NTN over NB-IoT. This proposal assumes exactly that network. The question it asks is what traffic should cross it.

The obvious design is the bent pipe: downlink everything and train on the ground. A LEO satellite completes an orbit in roughly 90 to 120 minutes and is in view of any single ground station for 5 to 15 minutes of it, but short passes alone do not justify computing in orbit. A global ground network adds passes, a polar station such as Svalbard can see nearly every orbit of a sun-synchronous satellite, and with cross-

links a satellite that has lost its ground contact can hand its data to a neighbor that still has one. Why, then, train in space at all? Three constraints answer the question, and relaying addresses none of them.

The first is volume, not timing. Relay changes when bytes reach the ground, not how many must. A constellation’s aggregate downlink is bounded by the number and rate of its gateway links, and for optical links by clear sky over a handful of ground telescopes, while sensing volume grows with every sensor generation; Denby and Lucia built the case for orbital edge computing on this arithmetic, showing that a nanosatellite camera fills its downlink many times over [2]. Even with a pass on every orbit, a satellite is in contact for about a tenth of its orbit, and each pass still lasts about ten minutes. One Earth-observation satellite produces gigabytes to terabytes of imagery per day; a ResNet-18 update is about 45 MB before compression. Federated learning moves the second quantity instead of the first, a reduction of two to four orders of magnitude that holds whatever the topology.

The second is energy per byte per hop. Every relay hop is a transmit and a receive on a bus with 1 to 150 W to spend [3], and a relay through GEO faces roughly 30 dB more free-space loss than a direct pass at 1,000 km slant range, which is why small satellites cannot close that link at useful rates. Relaying raw imagery multiplies its energy cost by the hop count; relaying a 45 MB update through three hops costs little. FL cuts bytes at the source, before the first hop, which lowers the cost of every hop after it.

The third is data that cannot or should not leave the platform. Ground stations sit in specific jurisdictions, and imagery that is export-controlled, commercially restricted, or owned by another operator in a shared constellation may be barred from crossing them; model updates can travel where pixels cannot. Even where the data may leave, the frames most worth learning from are the ones the bent pipe would drop. Roughly two-thirds of the Earth is under cloud at any time [4], so an on-orbit cloud-screening or scene-selection model is the first thing an operator wants, and the imagery that model must adapt to (this sensor, this orbit’s illumination, this season’s coverage) is precisely the imagery not being downlinked. Training that model where the data is, and sharing only what generalizes, is what FL does.

None of this retires the bent pipe. A large institutional satellite with EDRS-class relay, a global ground network, and no sovereignty constraint can downlink everything and train on the ground, and it should. This proposal targets the other regime: small-satellite fleets whose sensing outruns their downlink budget, or whose raw data cannot leave the platform. That regime grows with every camera-equipped small satellite, and within it the payoff of FL is established [5, 6, 7, 8, 9, 10]. We take that payoff as a premise and turn to the question the literature has not asked: what systems substrate satellite FL should run on.

The substrate under satellite FL today was built for datacenters. Every published framework assumes an always-on parameter server and a persistent training runtime on every satellite [5, 6, 8, 9, 10]. Between contacts that runtime holds memory and draws on the power budget the payload needs, while the ground-side server idles for hours between aggregations; published training runs are measured in days [5]. Serverless computing removes exactly these idle costs. A Function-as-a-Service (FaaS) platform invokes code on events and scales to zero between them, which is what an orbital duty cycle already looks like. Yet serverless FL has been studied only in terrestrial clouds [11, 12, 13, 14], where its three founding assumptions hold: clients are always reachable, bandwidth is stable, and compute is elastic. A LEO constellation violates all three.

The relay network is the setting for this work, not an alternative to it. Cross-link contacts follow from orbital mechanics as exactly as ground passes do, and for satellites in different planes they are just as intermittent. The sink-satellite and hierarchical aggregation patterns studied in Research Question 2 are the relay architecture; the open question is where training state lives while updates traverse it. Serverless and FL each fit that setting on their own. Their combination breaks, because FaaS statelessness and scale-to-zero collide with FL’s iterative training state and with connectivity that is intermittent but predictable. Three sub-problems follow, and each maps onto one research question.

(P1) Cold starts against short contact windows. A FaaS cold start for a function that must load a training framework and a model takes seconds to tens of seconds, a real fraction of a 5 to 15 minute window. In a

terrestrial cloud, request arrival is stochastic, so pre-warming is necessarily a prediction [15, 16]. In orbit, arrival is deterministic: SGP4 propagation of public two-line elements (TLEs) places a satellite within about 1 km at epoch. No serverless platform exploits this.

(P2) Where aggregation state lives. FL aggregation is stateful; the global model persists across rounds. FaaS is stateless by design. A satellite has no stable external state store within reach, and ISL bandwidth is scarce. Whether orbital aggregation should be stateless (re-fetch the global model on each invocation), stateful (pin it on a sink satellite), or a hybrid has never been measured.

(P3) Training state stranded mid-round. When a satellite leaves visibility partway through a round, its in-progress state (weights, optimizer moments, epoch counter) is stranded. Existing migration systems [17, 18] move generic service state unconditionally. Training state is different. Carrying Adam’s second-moment estimates onto a node whose data distribution differs can bias those estimates and slow or destabilize convergence. Which state is worth moving, and when, is a question about FL semantics that no migration mechanism answers.

2 Differentiation from the Current State-of-the-Art

No published or preprinted system combines serverless execution with federated learning on a satellite constellation. We reached that conclusion through a systematic survey of more than 40 systems across the three pairwise intersections, current as of mid-2026. Each body of work below supplies one ingredient and lacks the other two.

Terrestrial serverless FL. The TU Munich line established FaaS-based FL. FedLess [11] runs clients and aggregators as functions across heterogeneous providers; FedLesScan [12] adds clustering-based straggler mitigation; Apodotiko [13] adds scoring-based client selection under hardware heterogeneity. On the server side, λ -FL [19] and Just-in-Time Aggregation [20] made aggregation serverless, LIFL [14] is the current state of the art with an eBPF and shared-memory hierarchical data plane, and FLStore [21] moved FL’s non-training data plane onto serverless storage. All of these assume a reachable control plane, stable bandwidth, and elastic compute. None models intermittent connectivity, mobility, or an energy budget, and each handles cold starts by heuristic. FedLess is our software base, and it is also the clearest statement of what a terrestrial serverless FL system takes for granted.

Satellite FL without serverless. FedSpace [5] schedules aggregation around deterministic orbital connectivity and formalizes the idleness-staleness trade-off. Razmi, Matthiesen, Dekorsy, and Popovski developed ground-assisted asynchronous FL, visibility-aware scheduling, and energy-aware computation scheduling that triples battery lifetime without convergence loss [6, 7, 8]. AsyncFLEO [9] and FedLEO [22] use intra-orbit propagation and sink satellites. FedSN [23] and SatFed [24] handle heterogeneity and constrained links. Kim, Svoboda, and Lane [10] evaluated off-the-shelf FL in orbit across 768 configurations and state that they do not address serverless infrastructure. Every one of these systems assumes a persistent runtime on each satellite and an always-on parameter server. This literature also fixes the evaluation methodology we adopt: simulated orbits driving EuroSAT- and CIFAR-class workloads.

Serverless on the LEO edge without FL. TU Berlin’s Komet [17] runs FaaS with a replicated state store on the LEO edge and migrates services so that clients see virtual stationarity. Trabant [3] adds multi-tenant FaaS under energy and thermal envelopes and lists FL as future work. Celestial [25] is the same group’s microVM testbed. Krios [18] contributes proactive orchestration and handover for a LEO compute cloud. TU Wien’s HyperDrive [26] schedules serverless functions across the edge-cloud-space continuum with SLO awareness, and Databelt [27] propagates generic workflow state along that continuum. None of these runs FL. There is no aggregation, no round structure, no convergence to protect, and the state they move carries no training semantics.

Cold-start scheduling. The survey by Golec et al. [15] covers the cold-start literature, and Azure’s hybrid

histogram policy [16] is the production reference point. Both assume stochastic or learned arrival processes. No published policy exploits fully deterministic arrivals, which is what orbital mechanics provides.

The closest work to our third research question is EdgeFlow [28], which migrates a model sequentially between serverless functions in a terrestrial edge network and supplies a convergence analysis. It does not model orbital visibility, energy, or ISL topology, and it migrates unconditionally rather than deciding which state to carry. Against all four bodies of work, our differentiation rests on results that an FL-agnostic serverless-space paper cannot contain: a keep-warm policy proved optimal under deterministic arrivals, with a competitive bound against the best stochastic policy, and a convergence analysis that says when optimizer state should roam, reset, or transfer partially under non-IID data and bounded staleness. TU Berlin, TU Wien, and TU Munich are each one step from this intersection. That argues for speed, and for anchoring novelty in FL-specific proofs rather than in a deployment story.

3 Use cases

On-orbit model training for camera-carrying satellite fleets. EYE-class satellites carry a full-frame Sony sensor and produce far more frames than a ten-minute pass can carry. Suppose a fleet of such satellites trains a cloud-screening or scene-classification model federatively. Each satellite fine-tunes on its own frames during sunlit idle periods, an OrbitFaaS function wakes only when the SGP4 oracle reports that a pass or an ISL neighbor is imminent, and only the model update crosses the link. Raw frames stay on the satellite unless the model flags them as worth sending, so the downlink queue is ordered by value rather than by capture time, and the power draw of training stays inside the satellite’s eclipse budget. The same pattern applies to any Sony sensor flown on a third-party bus.

An NTN sensing continuum across satellites, high-altitude platforms, drones, and ground. Sony’s NTN footprint spans the ALT1250 modem on the ground, Airpeak drones in the air, and SOLISS-class optical terminals in orbit. An FL job that spans those tiers sees links that come and go on schedules: a drone’s flight plan, a platform’s orbit, a laser terminal’s pointing window. OrbitFaaS takes those schedules as input. It pre-warms an aggregator on the tier that will be reachable next, chooses stateful aggregation while a persistent optical chain exists and stateless aggregation across transient passes, and carries or drops optimizer state at each handoff according to the OrbitPath-FL policy. The result is a software layer that schedules around the intermittency Sony’s NTN hardware already lives with.

4 Methods

Three research questions share one testbed. The 12-month plan delivers Research Questions 1 and 2 in full and runs Research Question 3 as a gated pilot; if the gate passes, the pilot expands in the final quarter.

4.1 Research Question 1: OrbitFaaS, visibility-aware cold-start scheduling

A visibility oracle propagates public TLEs with SGP4 (via Skyfield) and emits, for every satellite and ground station, the sequence of contact windows for the coming days. An OrbitController above Knative, with OpenFaaS as a second target, consumes that schedule and issues pre-warm, keep-warm, and scale-down decisions for FedLess client and aggregator functions [11].

With arrival times known, the keep-warm decision at each window boundary reduces to a comparison: the energy cost of holding a function warm through the gap to the next contact against the cost of a cold start at that contact. The offline optimum is exact, and we derive it in closed form as a function of gap length, cold-start latency, idle power, and window length. The interesting quantities are how much the best online policy loses relative to that optimum, which we bound with a competitive analysis against Azure’s hybrid

histogram [16], and how fast the optimum degrades as SGP4 error grows. TLE propagation is accurate to about 1 km at epoch and drifts 1 to 3 km per day, so we sweep prediction error explicitly rather than assume it away.

Two evaluation elements answer the objections we expect. An ablation separates savings that come from cold-start management from savings that come from FL-round scheduling, since FedSpace [5] and Razmi et al. [6, 7, 8] already exploit orbital determinism for the latter. A scale sweep from 6 to 100 satellites reports missed-contact ratio, time-to-first-update, p95 function start latency, and regret against the oracle. Figure 1 shows the architecture.

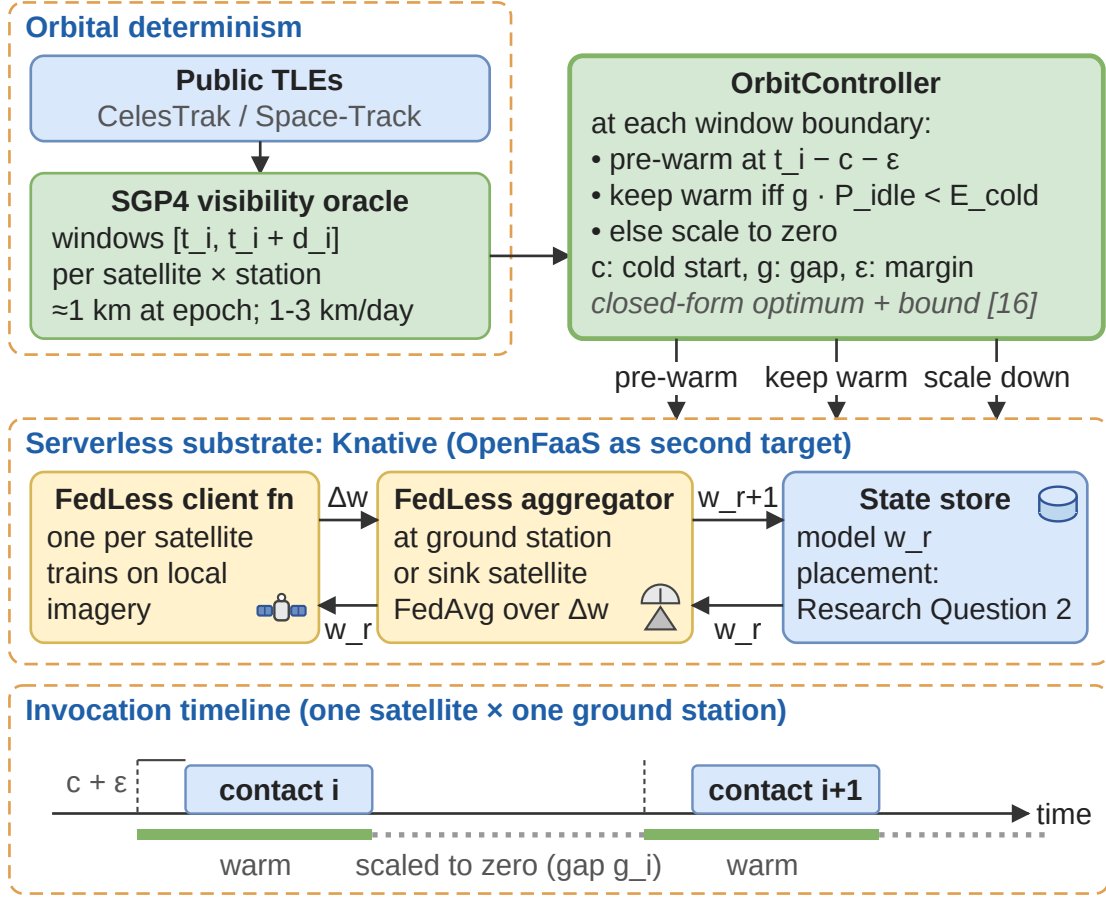


Figure 1: OrbitFaaS architecture: the SGP4 visibility oracle feeds the OrbitController, which issues pre-warm, keep-warm, and scale-down decisions over Knative for FedLess client and aggregator functions.

4.2 Research Question 2: aggregation-state placement in orbital FL

We implement four aggregation patterns on the testbed: pure stateless, where each aggregator invocation re-fetches the global model from a replicated store; pure stateful, where the model is pinned on a sink satellite; hierarchical, with intra-plane aggregation before a cross-plane merge; and gossip-stateless. All four run on Starlink Shell-1 topologies with fault injection (satellite loss, link drops) and node churn, under an energy model that follows real-satellite measurements [29] and the energy-aware FL results of Razmi et al. [8]. We measure each pattern on energy per useful update, bytes crossing ISLs, staleness distribution, recovery time after a failure, and state read and write amplification.

The measurements feed a selector. We formalize pattern selection as choosing, per aggregation epoch, the pattern that minimizes a weighted cost over energy, ISL bytes, expected staleness, and recovery time subject to the round deadline, and we fit the selector to the trade-off matrix. The hypothesis is that a regime boundary exists and that a hybrid policy, roughly stateless on visibility passes and stateful along persistent ISL chains, beats both pure patterns at once. If the boundary turns out to be trivial we will say so. The formalized selector and the fault-injection results are what separate this from a measurement study. See Figure 2.

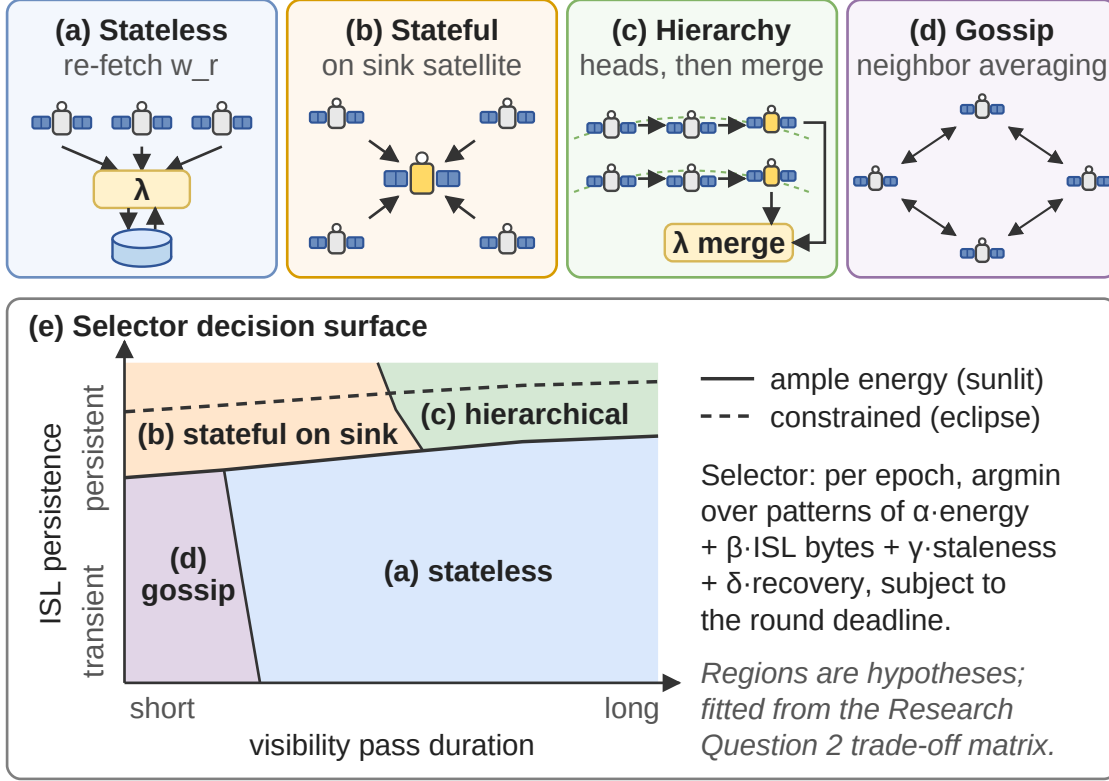


Figure 2: The four aggregation patterns and the selector’s decision surface over visibility duration, ISL persistence, and energy budget.

4.3 Research Question 3 (gated pilot): OrbitPath-FL, convergence-aware state handoff

When a satellite leaves visibility mid-round, the handoff engine picks one of three actions: roam (carry model and full optimizer state), re-init (carry the model, reset the optimizer), or partial transfer (carry the model and the first moment, quantized). Its inputs are the client’s remaining local work, the transfer budget the oracle predicts for the closing window, and a staleness estimate. CRIU checkpoint and restore is the primary mechanism; a WebAssembly runtime is an ablation, since no in-orbit Wasm deployment has been demonstrated. The analysis builds on the SCAFFOLD and FedOpt toolkit to establish, under non-IID data and bounded staleness, when each action preserves convergence. Pollux [30] is the closest analogue in spirit: it schedules by goodput rather than raw throughput, and OrbitPath-FL decides by expected convergence contribution rather than by bytes moved.

The gate is pre-committed. A pilot on CIFAR-10 and EuroSAT with three seeds and one non-IID degree runs in Q3. The full study proceeds only if the adaptive policy beats the best fixed policy (always-roam,

always-reinit, always-partial) by more than seed variance with no loss in final accuracy. Otherwise the claim narrows to competitive accuracy at lower wall-clock, and Q4 goes to robustness studies for Research Questions 1 and 2.

4.4 Evaluation substrate

Orbital simulation drives a real serverless backend running a real FL framework. Hypatia [31] and StarryNet produce orbital and network traces from public Starlink TLEs; Celestial [25] emulates satellites as microVMs; FedLess [11] and Flower run on Knative. Only the orbital and RF layer is simulated, which is standard practice in this literature [5, 6, 7, 8, 9, 10]. Baselines follow Section 4: FedLess, FedLesScan, Apodotiko, and LIFL for serverless FL; FedSpace, the Razmi schedulers, AsyncFLEO, FedSN, and SatFed for satellite FL; Komet, Krios, and EdgeFLOw for state migration; the Azure histogram policy for cold starts.

EuroSAT [32] is the primary benchmark: 27,000 labeled Sentinel-2 patches, 10 classes, 13 spectral bands, 64 by 64 pixels at 10 m, trained with CNN, ResNet-18, and EfficientNet-Lite class models. MNIST and CIFAR-10 serve cheap ablations. One large Earth-observation dataset, a fixed subset of BigEarthNet (about 590,000 Sentinel-2 and Sentinel-1 patch pairs with 19-class multi-labels) or So2Sat LCZ42, appears in at least one run so the results are not confined to small datasets.

Across research questions we report time-to-target-accuracy, cold-start hit rate, wasted warm-keep energy, missed-contact-window ratio, p95 function start latency, scheduler regret against the oracle, bytes per useful update, staleness distributions, recovery time after link or node failure, energy-normalized accuracy gain, checkpoint and restore latency, and handoff success rate.

Our targets are conditioned and falsifiable. For Research Question 1 we expect a 2 to 5x reduction in wasted warm time and in time-to-first-update at 100-satellite scale against histogram pre-warming, holding while SGP4 error stays within a few kilometers; we will report the error level at which the advantage disappears. For Research Question 2 we expect the hybrid selector to match the better pure pattern on time-to-accuracy while cutting ISL bytes or energy per update by a measurable margin at Shell-1 scale. Both figures are estimates, and we will report the regime boundaries where each gain vanishes.

The contributions sit at the systems and scheduling layer, so small models suffice. Research Questions 1 and 2 are mostly CPU work with light GPU use; Research Question 3’s convergence sweeps are the dominant cost and fit one to two GPUs over several weeks. The group’s STARFed work [33] ran on a single workstation (an i9-12900K, 32 GB of RAM, and one RTX 3090) with MNIST, CIFAR-10, and EuroSAT, and this project is sized the same way. Purdue’s Gilbreth GPU cluster, operated by the Rosen Center for Advanced Computing, is available as overflow for the Research Question 3 sweeps.

Prof. Bhargava’s group published STARFed, a link-aware defense against poisoning in satellite-terrestrial FL [33], and has a long record in distributed systems and security. Xiao Luo, the student lead, comes from serverless systems research, has FedLess running, and completed the survey of more than 40 systems and the evaluation design summarized above.

5 Goals / Milestones

G1: Deliver OrbitFaaS with a keep-warm policy proved optimal under deterministic arrivals and a competitive bound against stochastic pre-warming. G2: Characterize aggregation-state placement on Shell-1-scale topologies and validate the hybrid selector. G3: Run the OrbitPath-FL gating pilot and, if it passes, the convergence study. G4: Release the orchestrator, the testbed configuration, and the trained models as open source.

The project produces an open-source orchestrator and a reusable testbed that couples orbital simulation to a serverless FL stack, one or two submissions to MLSys, EuroSys, or SoCC-class venues, a character-

Quarter	Months	Activities	Deliverable
Q1	1-3	Stand up the shared testbed (Celestial, Hypatia/StarryNet, Knative, FedLess/Flower, Starlink Shell-1 TLEs). Implement the four aggregation patterns and the orbital energy model.	Quarterly Report 1
Q2	4-6	Complete the Research Question 2 trade-off matrix (pattern x constellation scale x churn x non-IID level). Derive and validate the hybrid selector. Begin the SGP4 visibility oracle and OrbitController.	Quarterly Report 2
Q3	7-9	Complete Research Question 1: closed-form keep-warm policy, competitive bound, full evaluation with the cold-start versus FL-scheduling ablation. Run the Research Question 3 gating pilot.	Quarterly Report 3
Q4	10-12	Expand Research Question 3 if the gate passes, otherwise deepen Research Question 1 and 2 robustness studies. Integrate results. Open-source release of OrbitFaaS and testbed artifacts. Submit to an MLSys, EuroSys, or SoCC-class venue.	Final Research Summary Report

Table 1: Proposed project schedule

ized aggregation-state design space with the fitted selector, and EuroSAT and BigEarthNet checkpoints for reproducibility. Sony receives three quarterly reports and a final research summary on Table 1 above.

References

- [1] Japan Aerospace Exploration Agency, National Institute of Information and Communications Technology, and Sony Computer Science Laboratories, Inc. Small optical link for international space station (SOLISS) succeeds in bidirectional laser communication between space and ground station. JAXA Press Release, Apr. 23, 2020. [Online]. Available: https://global.jaxa.jp/press/2020/04/20200423-1_e.html, 2020.
- [2] Bradley Denby and Brandon Lucia. Orbital edge computing: Nanosatellite constellations as a new class of computer system. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.
- [3] Tobias Pfandzelter, N. Bauer, A. Leis, C. Perdrizet, F. Trautwein, Trever Schirmer, Osama Abboud, and David Bermbach. Trabant: A serverless architecture for multi-tenant orbital edge computing, 2025.
- [4] Michael D. King, Steven Platnick, W. Paul Menzel, Steven A. Ackerman, and Ping Yang. Spatial and temporal distribution of clouds observed by MODIS onboard the Terra and Aqua satellites. *IEEE Transactions on Geoscience and Remote Sensing*, 51(7):3826–3852, 2013.
- [5] Jinhyun So, Kevin Hsieh, Behnaz Arzani, Shadi A. Noghabi, Salman Avestimehr, and Ranveer Chandra. FedSpace: An efficient federated learning framework at satellites and ground stations, 2022.
- [6] Nasrin Razmi, Bho Matthiesen, Armin Dekorsy, and Petar Popovski. On-board federated learning for dense LEO constellations. In *IEEE International Conference on Communications (ICC)*, 2022.

- [7] Nasrin Razmi, Bho Matthiesen, Armin Dekorsy, and Petar Popovski. Scheduling for ground-assisted federated learning in LEO satellite constellations. In *European Signal Processing Conference (EUSIPCO)*, 2022.
- [8] Nasrin Razmi, Bho Matthiesen, Armin Dekorsy, and Petar Popovski. Energy-aware federated learning in satellite constellations. In *IEEE Global Communications Conference Workshops (GLOBECOM Workshops)*, 2024.
- [9] Mohamed Elmahallawy and Tie Luo. AsyncFLEO: Asynchronous federated learning for LEO satellite constellations with high-altitude platforms. In *IEEE International Conference on Big Data (BigData)*, 2022.
- [10] Grace Kim, Filip Svoboda, and Nicholas D. Lane. Bringing federated learning to space. In *IEEE Aerospace Conference*, 2026.
- [11] Andreas Grafberger, Mohak Chadha, Anshul Jindal, Jianfeng Gu, and Michael Gerndt. FedLess: Secure and scalable federated learning using serverless computing. In *IEEE International Conference on Big Data (BigData)*, pages 164–173, 2021.
- [12] Mohamed Elzohairy, Mohak Chadha, Anshul Jindal, Andreas Grafberger, Jianfeng Gu, Michael Gerndt, and Osama Abboud. FedLesScan: Mitigating stragglers in serverless federated learning. In *IEEE International Conference on Big Data (BigData)*, pages 1230–1237, 2022.
- [13] Mohak Chadha, Alexander Jensen, Jianfeng Gu, Osama Abboud, and Michael Gerndt. Apodotiko: Enabling efficient serverless federated learning in heterogeneous environments. In *IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2024.
- [14] Shixiong Qi, K. K. Ramakrishnan, and Myungjin Lee. LIFL: A lightweight, event-driven serverless platform for federated learning. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- [15] Muhammed Golec, Guneet Kaur Walia, Mohit Kumar, Félix Cuadrado, Sukhpal Singh Gill, and Steve Uhlig. Cold start latency in serverless computing: A systematic review, taxonomy, and future directions. *ACM Computing Surveys*, 57(3), 2024.
- [16] Mohammad Shahradd, Rodrigo Fonseca, Íñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *USENIX Annual Technical Conference (ATC)*, pages 205–218, 2020.
- [17] Tobias Pfandzelter and David Bermbach. Komet: A serverless platform for low-earth orbit edge services. In *ACM Symposium on Cloud Computing (SoCC)*, pages 866–882, 2024.
- [18] Vaibhav Bhosale, Ada Gavrilovska, and Ketan Bhardwaj. Krios: Scheduling abstractions and mechanisms for enabling a LEO compute cloud. In *ACM Symposium on Cloud Computing (SoCC)*, pages 322–340, 2024.
- [19] K. R. Jayaram, Vinod Muthusamy, Gegi Thomas, Ashish Verma, and Mark Purcell. λ -FL: Serverless aggregation for federated learning. In *International Workshop on Trustable, Verifiable and Auditable Federated Learning (FL-AAAI-22)*, 2022.
- [20] K. R. Jayaram, Ashish Verma, Gegi Thomas, and Vinod Muthusamy. Just-in-time aggregation for federated learning. In *IEEE International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2022.
- [21] Ahmad Faraz Khan, S. Fountain, Ahmed M. Abdelmoniem, Ali R. Butt, and Ali Anwar. FLStore: Efficient federated learning storage for non-training workloads. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
- [22] Z. Zhai, Q. Wu, S. Yu, R. Li, F. Zhang, and X. Chen. FedLEO: An offloading-assisted decentralized federated learning framework for low earth orbit satellite networks. *IEEE Transactions on Mobile Computing*, 23(5):5260–5279, 2024.
- [23] Zheng Lin, Zhe Chen, Zihan Fang, Xianhao Chen, Xiong Wang, and Yue Gao. FedSN: A federated learning framework over heterogeneous LEO satellite networks. *IEEE Transactions on Mobile Computing*, 2024.
- [24] Y. Zhang, Zheng Lin, Zhe Chen, Zihan Fang, W. Zhu, Xianhao Chen, J. Zhao, and Yue Gao. SatFed: A resource-efficient LEO satellite-assisted heterogeneous federated learning framework, 2024.

- [25] Tobias Pfandzelter and David Bermbach. Celestial: Virtual software system testbeds for the LEO edge. In *ACM/IFIP International Middleware Conference (Middleware)*, pages 69–81, 2022.
- [26] Thomas Pusztai, Cynthia Marcelino, and Stefan Nastic. HyperDrive: Scheduling serverless functions in the edge-cloud-space 3D continuum. In *IEEE/ACM Symposium on Edge Computing (SEC)*, 2024.
- [27] Cynthia Marcelino, L. Guelmino, Thomas Pusztai, and Stefan Nastic. Databelt: A continuous data path for serverless workflows in the 3D compute continuum, 2025.
- [28] Y. Shi, Q. Hou, P. Fan, and Khaled B. Letaief. EdgeFlow: Serverless federated learning via sequential model migration in edge networks, 2026.
- [29] Q. Li, S. Wang, C. Xu, X. Ma, M. Xu, A. Zhou, R. Xing, B. Yang, Z. Zhu, Y. Zhang, and X. Liu. Exploring real-time satellite computing: From energy and thermal perspectives. In *IEEE Real-Time Systems Symposium (RTSS)*, pages 161–173, 2024.
- [30] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R. Ganger, and Eric P. Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021.
- [31] Simon Kassing, Debopam Bhattacharjee, André Baptista Águas, Jens Eirik Saethre, and Ankit Singla. Exploring the “Internet from space” with Hypatia. In *ACM Internet Measurement Conference (IMC)*, pages 214–229, 2020.
- [32] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7):2217–2226, 2019.
- [33] Z. Liu, Bharat K. Bhargava, and Nagender Aneja. STARFed: Link-aware defense against poisoning attacks in satellite-terrestrial federated learning. *IEEE Transactions on Network Science and Engineering*, 13:3053–3069, 2026.

Budget Summary

We request \$100,000, the Faculty Innovation Award ceiling, for 12 months. The award is all-inclusive, so Purdue's indirect costs are contained within this total rather than added to it (See Table 2).

Category	Description	Rate / Effort	Amount (USD)
Salaries			
PD/PI: Bharat Bhargava	Academic Year (Base: \$354,538)	0.00 Months (0.00%)	\$0
PD/PI: Bharat Bhargava	Summer (Base: \$384,082)	1.52 Months (12.66%)	\$48,643
Total Salaries			\$48,643
Fringe Benefits			
Academic Year		26.83%	\$0
Summer		26.83%	\$13,051
Total Salary & Wages			\$61,694
Other Direct Costs			
Travel	Domestic		\$2,000
Total Direct Costs			\$63,694
Indirect Costs	Purdue Indirects (MTDC Base: \$63,694)	57.00%	\$36,306
Total Costs	(Basic Research Total)		\$100,000

Table 2: Budget summary of this proposal

Purdue overhead is included in the \$100,000 total and is not requested in addition to it. This proposal contains no confidential or proprietary information; every cited result is public.